

# PyBe Feature Proposal

## What is the feature?

- Design and sequences to teach **recursion**.
- **Recursion** will be first taught as a general problem-solving pattern (fully independent of programming), - and only later, it will be connected to programming.
- The learner will be introduced to **recursion** through 6 different stages :
  1. starting at making the learner feel the need to have a better way to solve a problem
  2. introducing the **recursion** way of solving problem without revealing the name
  3. introducing the actual concept **recursion**
  4. its application in real-world scenarios
  5. connecting it to programming through simple problems like finding factorial, finding sum of natural numbers, etc
  6. finally showing the application of **recursion** in solving complex problems in computing/programming.

## Stages in putting forth the idea of recursion

### Stage 1 — Feel the need

1. It will have a real-world problem and it will be solved using naive/obvious approach such as loop/repetition.
2. Focus is to show the learner that these naive/obvious approach became tedious , and there is a need for having a new approach to solve this problem.

### Stage 2 — Show the new approach

1. The idea of new approach will be presented to students, and the same problem from stage 1 will be solved using this new approach.
2. The focus will be to show that how the new approach solved the same problem in much simpler way .
3. So by the end of this stage, the student will have the intuition of "recursion" (the actual concept will still remain unnamed).

### Stage 3 — Reveal the concept

1. Recursion will be presented as a concept, and the main idea behind it.
2. There will be some case-studies/activities/stories to teach the following aspect of recursion :
  - How recursion works.
  - What kinds of problems it suits.
  - What kinds of problems it probably shouldn't be used for.
  - Delegating step and stopping point (recursive step/ base case).
3. The focus is to put forth the entire concept of recursion.

### Stage 4 — A satisfying real-world application

1. It will contain one full, real-world scenario demonstrating the usage of recursion from end to end.

2. Every concept introduced in Stage 3, will be explicitly stated as where it appears in this one worked example.
3. The focus is leave the learner with a deep, settled understanding of the concept before bringing recursion in programming.

### **Stage 5 — Connect to programming**

1. It will apply recursion to problems the learner can already solve another way — factorial, sum of natural numbers. This is to show as how the same problem can be solved using the principles of recursion.
2. Aim is to map the already-understood pattern of recursion onto programming terms.

### **Stage 6 — Where it's the real deal**

1. Contain the problem where recursion is the only practical way to solve them cleanly — DFS, maze-solving.
2. It will be also explicitly stated as why other method such as loop proves to be failure in this case, and why recursion is the only feasible way.
3. Closes the recursion topic by showing the real importance of recursion, not just being an interesting alternative.

## **Why recursion is chosen as the feature ?**

1. The idea of recursion is "to solve a problem by delegating a smaller version of it, and waiting on that smaller version's answer." But when recursion is introduced in classes, it is introduced as a concept of programming, and as a another method to solve the problems which the students already knows how to solve. This fails to present recursion as an idea independent from programming, and hides the actual need for it. This feature will try to present **recursion** through discovery-driven approach and try to show the genuine need for it.
2. There is a common misconception among CS students regarding recursion. Recursion is seen as the **"looping model"** (mistaking recursion for repetition) instead of correct **"copies model"** (each call as a distinct instance, waiting on a smaller one to resolve). This feature will try to put forth the idea of recursion without any code or programming vocabulary, so that the idea of recursion is not seen as primarily as a loop construct.